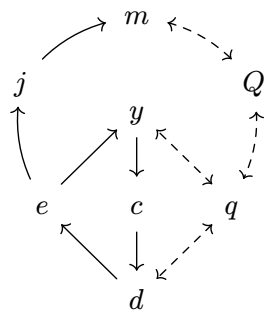
D I Z Z Y

A Philosophy and Schema System for the Development of Certain Event Driven Architectures with Deferred Interstitial Infrastructure

Conrad Mearns

2026-04-26



> "The required techniques of effective reasoning are pretty formal, but as long as programming is done by people that don't master them, the software crisis will remain with us and will be considered an incurable disease."
>
> — Edsger W. Dijkstra, 2000

Abstract

Software projects fail in predictable ways. Infrastructure decisions made under pressure harden into permanent constraints. Domain logic becomes inseparable from the languages that encode it. Data and services become inseparable from the databases that serve. The vocabulary used to describe a system drifts from the code that implements with every commit, requiring continuous and careful maintenance. DIZZY is a philosophy for building event-driven software systems that keeps domain logic, data contracts and infrastructure permanently and deliberately separate. It draws on established practices - Command Query Responsibility Separation, Event Sourcing, and Domain-Driven Design - and composes them into a coherent discipline: define the domain in a language-agnostic schema, generate typed contracts for data and code, and let infrastructure decisions follow rather than lead. This paper is written for software practitioners and technical-decision-makers who have felt these problems firsthand. It makes the case for why the separations DIZZY enforces matter, introduces a system of eight program components that illustrates the philosophy, and describes a software generation pipeline that closes the gap between domain discovery and working, scalable software.

Table of Contents

1. The Burden of Software Architecture is…

1.1. The Irreversibility of Poor Decisions

Everything in computing changes - it's just a matter of *when* the change occurs. Our demands change, our platforms, capabilities, problems, fashions, and scale all change too.

We often see, especially in the most cursed software projects, that poor decisions will rear their heads to haunt us again later on.

Decisions that seemed either well intentioned, necessary, critical, or even genuinely brilliant can all turn into a curse when the conditions are right. And truly - I don't know what is worse: the number of projects people have retained as sunk costs, due to pride or politics - or the number of people who seemingly have no awareness of the curses they write.

(Though, hindsight punishes the early architects all too often. There are too many such cases, when a developer succumbs to the pressures of the labor. When they break, integrity breaks too. We see mitigations, shortcuts, hacks and sloppy writing apparent as the evidence of such mental slips and burnouts.)

A decent Software Architecture is a ward against such curses. It is the draft of constraints that prevent the workarounds, the dirty hacks, and the "clever" tricks that inevitably cause confusion and harm later. A decent architecture promotes modularity for the sake of repairability.

Poor Architectures are those that attract curses rather than features. They are those whose structure itself is vile physarum of traces that span dozens of files, and couples itself to hardcoded strings, special conditionals, and ancient rituals of practices and patterns of by-gone developers.

We call this "Coupling"

1.2. The False Dichotomies of Deployment Solutions

Every decision has the potential to become a permanent constraint. Microservices on monoliths, AWS or Azure, Svelte or Vue or React… The problem is not deciding which option is *correct* - the choice itself is what become load-bearing.

There are many stories of those who find a themselves in some peril with bad architecture, and how they saved themselves with a large and heroic refactor. These stories illustrate the fear and focus of Software Architecture. We can take direct comparisons against the prices others have paid for the wrong decision - and how long such mistakes took to fix.

Such is also the case for vendor lock-in - whether it be a Service maintained by another organization, or an entire cloud provider.

1.3. Applications as Islands

The isolation of data within applications hurts both consumers and businesses.

As a consumer, the data we generate from our photos, health integrations, recipes, liked posts, videos, journal entries etc are effectively useless to us. Power users of information cannot access their own data from the Walled Gardens of the internet without using complex and bespoke API's, and cannot integrate their applications together on their own.

Similarly, even within a single business teams struggle to share data effectively. The interfaces for which data is shared requires bespoke translations and jury-rigged events.

1.4. The Map Grossly Mislabels the Territory

The Map is Not the Territory. The map usually hardly comes close to the territory.

When leadership and business stakeholders make decisions about their software, it's critical that they understand the system. We still rely on slide decks, books of documentation, and tribal knowledge over code in many instance.

Why? Because the code teaches us nothing about how the code works, why the code works, and what decisions led to the code being created.

1.5. Scaling Software without Adequate Support

There will always be a natural limit to the number of experienced software developers who can take on the craft of design and implementation, as well as the discussion and delegation of work, and also ensure timeliness in the execution of a project.

The Standish CHAOS study finds that only 31% of software projects are successful, 52% are over budget or miss missed deadlines, and that 19% of projects fail completely. [1]

Melvin Conway states that systems designed by organizations are constrained to mirror the communication structures of those organizations. [2] The Inverse Conway Maneuver turns this into a design tool: deliberately shape team boundaries and system architecture together, so that communication structures produce the architecture you want rather than constrain it. [3]

As software begins, it's design only must necessitate the communication structure of the individual. As software grows, and it's design becomes becomes an argument of the individuals and the many. Boundaries are built around convince and practicality, rather than sustainability, maintainability.

As software scales to serve more, and it's features become uncountable, and the ability for developers, stakeholders, and leadership cannot retain the necessary mental models to make accurate predictions of change.

2. Software Architectures Should…

…deliberately shape the communication structures of the organizations that build them.

The Inverse Conway Maneuver says: if your system is going to mirror your organization anyway, design it to mirror the organization you *want*. DIZZY applies this as an architecture constraint. Its rigid component boundaries are deliberate seams that define where teams own work independently and where they must coordinate.

2.1. Defer Deployment Decisions

The best decision is often the one you don't have to make yet.

DIZZY is a a philosophy of software architecture that emphasizes the deference of choice.

Which databases, which message brokers, which languages and whatever other foundations that required are deferred until they are needed.

DIZZY prefers to answer and understand the business problems to solve from first principals first - and let the tradeoff space be explored at a different layer of the architecture.

2.2. Architect for Reversibility

Reversibility is about maintaining the freedom to respond to new information.

DIZZY should not only empower modularization of our system, it should naturally enable A/B testing and hotswapping so that reversing decisions becomes a standard practice, rather than a hail mary.

Consider the questions that haunt every long-lived system. What if the database vendor raises prices or discontinues the product? What if the chosen query model turns out to be wrong for the data? What if the deployment topology is generating too high of an operational cost?

In most architectures these are existential questions — answering them requires rewriting code that should never have known about infrastructure in the first place.

2.3. Support (Almost) Any Programming Language

Truly modular software should work across disciplines, languages, networks, and hardware. DIZZY achieves language independence through language-agnostic schemas and code genera-

tion. By generating interfaces and protocols for the DIZZY Processes, the bare minimum of guarantees can be made to show what components are capable of what tasks.

2.4. Separate Data from Process

DIZZY is intentionally a mostly functional paradigm. Data (commands, events, models, and query IO) and processes (procedures, policies, projections, and queriers) are explicitly separated. This is fundamentally different form the traditional Object Oriented Paradigm that traditional DDD is based upon.

This separation is what enables various components of the full DIZZY system to be deployed independently.

2.5. Derive Infrastructure *from* Code

DIZZY derives infrastructure requirements separate from the domain model.

Just as Programming Languages translate human intent to machine code - so too should DIZZY translate business intent to infrastructure. Automatically, with optimizations for infrastructure that can be refined out of phase with business needs - just as GCC can improve compilation and optimizations without requiring changes to C.

DIZZY calls this an Interstitial Infrastructure. The goal is to treat Interstitial Infrastructure as a compilation problem.

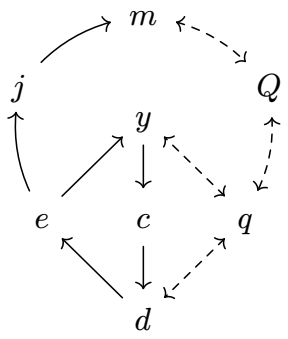
Infrastructure decisions are the final step in a DIZZY deployment, and enable DIZZY to be built as either a monolith, microservices, or something in between.

Commands, Events, and Models are data. They can be communicated over any number or combinations of message passing channels, such as ZeroMQ, Kafka, HTTP, WebSocket, gRPC, etc.

The deployment of Processes support any potential target as long as it can satisfy channel connectivity requirements.

3. DIZZY Solves this Using...

... four Data Contracts and four Program Processes.



DIZZY is Data Oriented and Functional. We explicitly outline each data contract component and each program process component separately. The diagram to the left shows the general data flow between each DIZZY data and process component. Solid arrows show message passing, dashed arrows show call and response communication.

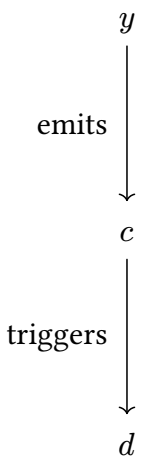
Symbol	Component	Role
c	Command	Represent intent - what a user or policy wants to happen
d	Procedure	Handles a Command and may emit Events
e	Event	Immutable record of facts - the source of truth
y	Policy	Reacts to an Event and may emit Commands
j	Projection	Listens for Events and updates Models
m	Model	Queryable view of state, derived from Events
Q	Querier	Executes a Query against a Model
q	Query	Typed contract for a call and its response

DIZZY is therefor comprised of two dataflow loops.

Firstly, a reactivity loop; Where Commands trigger Procedures, which emit Events that trigger Policies, that emit Commands. Secondly, a data retrieval loop; Where Event are projected to Models (databases), which can be queried by Procedures and Policies. The reactivity loop enables processing, algorithms, and business logic to react and change to new information. The retrieval loop enables database decoupling while providing efficient value lookups.

As an example, suppose we processing financial transactions. Each Events record debits and credits as the source of truth. Transactions are initiated by users and the data is represented by a Command. The Transaction Command triggers a procedure, which may need to Query for the normalized account balance to know whether to succeed or fail. Thus, each debit and credit Event must also trigger a projection to keep this normalized account balance up-to-date in a Model.

3.1. Commands (c) that Represent Intent



Commands represent expressions of intent, what users *want* to happen. They are ephemeral - not records of fact, and thus may be fire-and-forget and discarded. This distinction matters: intent can be rejected, retried, or rerouted.

Facts cannot.

Because Commands are ephemeral and may be discarded without consequence, the are useful for isolating the transmission of data must abide to different legal and business policies - such as personally identifiable information.

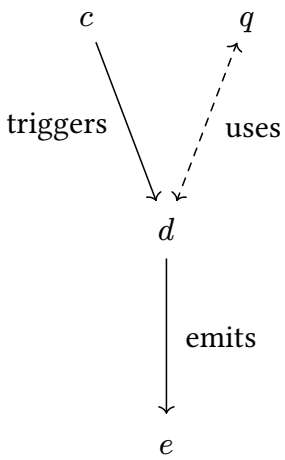
Whether this information is recorded as an immutable fact is up to the design of Procedures and Events.

A policy may emit any number of commands, of any command sub-types. Therefore policies and commands are one-to-many; several policies may emit the same command types.

A Command may trigger many Procedures. This relation is typically best assigned at the Procedure definition.

Commands express intent, and can be carefully designed to handle cases where lossy systems where retries are standard practice. See Durable Execution

3.2. Procedures (d) that Perform the Critical Work



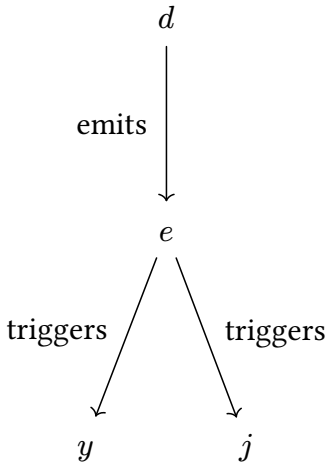
Procedures handle Commands and may emit Events. They are the site of critical work in a DIZZY system — the place where intent meets consequence.

Procedures are not required to be pure functions. They may query Models to inform their decisions. But any external system a Procedure touches must either be modelled as another DIZZY component, or its effects must be recorded in Events. This constraint is what makes Procedures portable across languages and deployable in any topology: their behavior is fully described by what they receive, what they query, and what they emit.

When a Procedure performs IO or causes effects in the world, it should emit Events to record the attempt, progress, and result. An effect that

is not recorded in an Event is invisible to the rest of the system — and invisible effects undermine the audit guarantee that Events provide.

3.3. Events (e) that are the Source of Truth



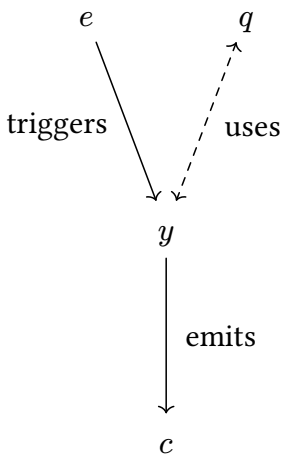
Events are the single source of truth. Everything else — every model, every view, every report — must be derived from events. Events are immutable records of what *happened*: once written, they cannot be altered or deleted.

Because events capture what happened independently of any particular database schema, the same event stream can power multiple databases with completely different schemas — each optimized for different queries. Different teams or services can maintain their own models from the shared event stream; coordination happens through the event contract, not through shared database access.

This also makes migrations tractable. Rather than transforming a live database in place, a new Model can be built alongside the old one from the same event history, and cut over when ready. If the new schema turns out to be wrong, the events remain — and the next attempt costs only a new Projection.

The event stream is the canonical representation. Every database is just a cached, queryable projection of that truth — disposable and rebuildable by design.

3.4. Policies (y) that React and Initiate

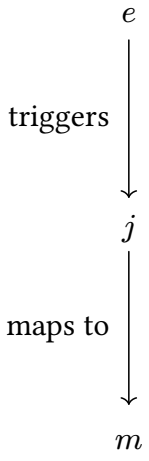


Policies are the reactive logic of a DIZZY system. Where Procedures respond to intent, Policies respond to fact: they are triggered by Events that have already occurred, and they may emit Commands in response. A Policy asks: *given that this happened, what should happen next?*

This distinction is not incidental. Because a Policy reacts to an Event — an immutable, already-happened fact — it is inherently decoupled from the Procedure that produced it. The Policy does not know or care how the Event came to exist. It knows only what happened, and responds accordingly.

Policies may also query Models to inform their decisions, allowing the system to react differently depending on current state. A Policy that always emits the same Command is a simple rule. A Policy that queries a Model first is a conditional one — but the conditionality lives in the Policy, not scattered through unrelated code.

3.5. Projections (j) that Map Events to Models

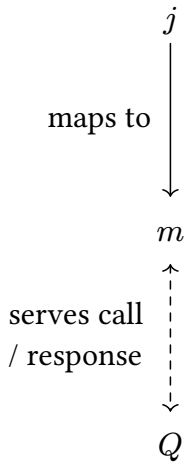


Events are the authoritative record of everything that has happened, but they are not optimized for retrieval. A Projection solves this: it listens for specific Events and updates one or more Models in response, translating the language of facts into the language of efficient queries.

This approach eliminates the need for Aggregates — the construct traditional Domain-Driven Design uses to bridge event-driven architecture to object-oriented state. Because Models in DIZZY are treated as ephemeral and rebuildable from Events, there is no need for an object that holds and manages state on behalf of a set of related entities.

The practical consequence is significant: if a Model turns out to be wrong for its purpose, it can be replaced without touching the event history. Write a new Projection, replay the Events, and a new Model emerges from the same source of truth.

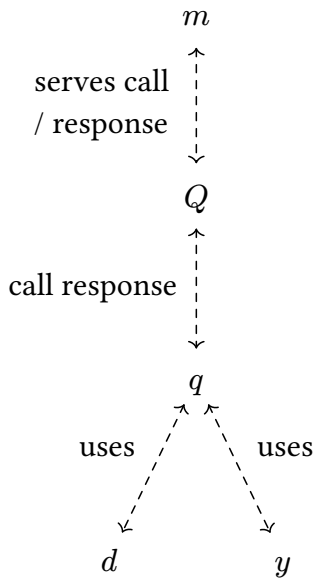
3.6. Models (m) that Serve Data for Queries



A Model is a schema for queryable state, backed by a database chosen for the specific access patterns that Model must serve. Models are derived from Events via Projections, and are rebuildable on demand.

Models exist to make queries fast, not to be the source of truth. You can have as many Models as you have distinct questions to answer — each backed by whatever database technology best suits those questions. A time-series store for metrics, a graph database for relationships, a search index for full-text: all derived from the same Event stream.

3.7. Queries (q) and Queriers (Q) for efficient computation



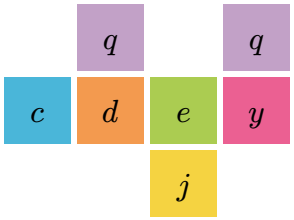
Querying can often be the most confused aspect of software engineering, because the retrieval of information tends to occur at the confluence of database requirements, data shape requirements, and downstream alignment.

DIZZY systems demand the distinction between database drivers, data contracts, and the program that 'executes' the query. We call the data contracts Queries, which are typically represented as a tuple of Input and Output, or Call and Response. The process that uses program code to execute the query is what we call the Querier.

4. How to Structure Software Development Workflows with DIZZY

DIZZY is a complete philosophy for software development; it covers the flow from ideation, vibe-checking, workshopping, integration, to production deployment. This whitepaper will cover the workshopping method, and less of vibe-checking. From there, it will show the tooling to illustrate what steps can be automated algorithmically and which need intervention.

4.1. Workshopping



DIZZY uses a workshop to bring business stakeholders, customers, scientists, and other domain experts together to find consensus around software processes. Computation and Systems experts should serve only to teach, ask questions, and suggest symbolic solutions to process gaps, NOT to define or suggest what would otherwise be the process flow.

The workshop is largely based on Event Storming [4] by Alberto Brandolini and uses many of the same techniques. Attendees write on sticky notes and place them on a wall next to other notes. Note colors are semantic symbols. Notes convey and narrate a system in the language of facts and intentions. Unlike Event Storming, DIZZY notes represent *real* system components.

Events ( ) pin the most important facets of the process, and are recorded in past tense: "order placed," "payment declined," "shipment dispatched." These records are facts by which we represent all state changes.

Commands ( ) hook process automation and users into the ecosystem by capturing an intent. They are written imperatively: "place order", "submit payment." These commands may represent a button, API call, a form, or any other method of supplying new instruction to a software system.

Procedures ( ), Policies ( ), Projections ( ), and Queries ( ), replace the "Aggregate", "View", and "Process/Policy" systems of the DDD-inspired Event Storming model.

After the workshop, the DIZZY workflow guides the progressive development of each component until a software artifact is produced. This allows stakeholders to build the software visually and without confounding details that aren't important to those stakeholders.

No database is chosen. No framework is selected. The output is a shared vocabulary in regards to the domain at hand.

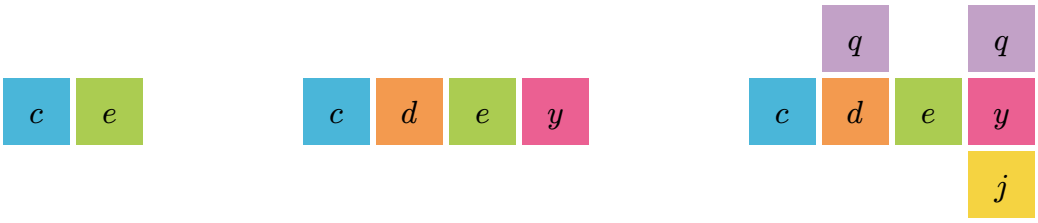
That vocabulary maps directly onto DIZZY's component schema. A workshop session does not merely produce a pile of forgettable photographs; it produces the skeleton of a *feature definition* which the rest of DIZZY transforms into functioning, scalable software.

This directness is intentional. DIZZY's eight components exist precisely so that the output of domain discovery can be written down without loss of meaning. A feature defined in DIZZY's vocabulary is still readable by the domain expert who participated in the session. The command names, event names, and policy reactions are their words - not a translation of them.

This couples the language of the Computationalists and the Domain Experts - allowing both to have a concrete and shared artifact that communicate bidirectionally between abstract graphs and charts - and deployed and running systems.

DIZZY borrows the collaborative spirit of Brandolini's original technique, but adapts the semantic and pragmatic functional paradigm. Aggregates, read models, and the object-oriented constructs of traditional Domain-Driven Design are absent. Every DIZZY component is either a data contract or a stateless process - another distinction that makes components independently deployable, testable, and replaceable in ways the original framing does not require.

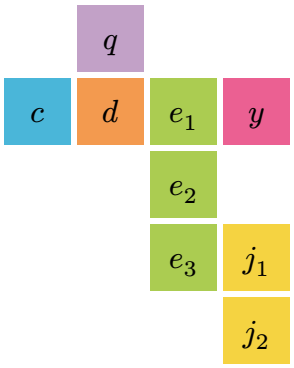
Like Event Storming though, the practice is intended to be simple and progressive. Start by pinning Commands and Events. This naturally exposes what procedures and policies may be required. In turn, this can expose new Commands and Events that my be required for consistency, which prompts more Procedures and Policies - etc.



When it is becomes time to define the gist of how Procedures and Policies function - this is where Projections and Queries come in.

Upon these notes, write the name of Model (representative of a database schema or ontology), and the name of the projection or query if it helps for disambiguation.

For instance, after an "Order Placed" Event, we may need to project the result to two Models - one for "Shipping", and another for "Inventory". Naming these projections further is optional at this phase, but is useful later when the projections require disambiguation.



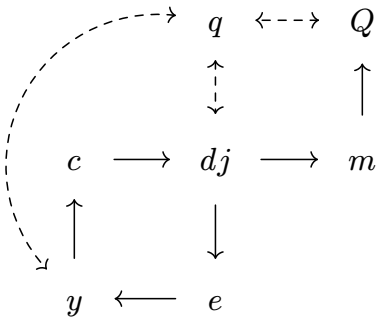
An exact semantics for organizing notes is not needed. If the workshop gets crowded - make copies of notes and spread things out until things make sense.

4.2. Studying Vibes and Experiments

DIZZY is hard to emulate with simple scripts, because a simple script is usually one that follows the UNIX Philosophy: file in, file out, do one job and do it well. We can fit the DIZZY components to fit the guise of UNIX by composing Procedures and Projections.

dj scripts sacrifice rigidity of the DIZZY core model to serve as an exercise of Intent and Event Extraction.

More on this: https://github.com/ConradMearns/without-objective/tree/main/Structured-Log-Pipes



Here, Q typically reflects a FileSystem. q represents read operations, while j represents writes. Policies (y) aren't explicitly required, but still encode some level of reactivity to events that have taken place.

The goal is *not* to write software like this. Rather, it's to identify that the UNIX philosophy was indeed the standard pattern for writing software, an explicit Structure Log Piping system expresses better flexibility in building small scripts, which can be more easily translated into scalable architectures later.

Using this abstraction, we can now peer into the black box which may be our script - and identify when and where changes to Models (file outputs, database calls) are made.

We can identify IO reads and assign them names as Queries, and we can begin to imagine what Events must exist in order to become DIZZY compatible.

In practice, this can be fairly simple. Often, it is enough to transform the base logging system into one which traces Side-Effects as DEBUG logs. When those logs are structured, and consistent, then we can begin to use the logs themselves as a sort of pseudo Change Data Capture / "Event Store" for projections.

This exercise allows for legacy software to be transformed to become DIZZY compatible.

4.3. Automation

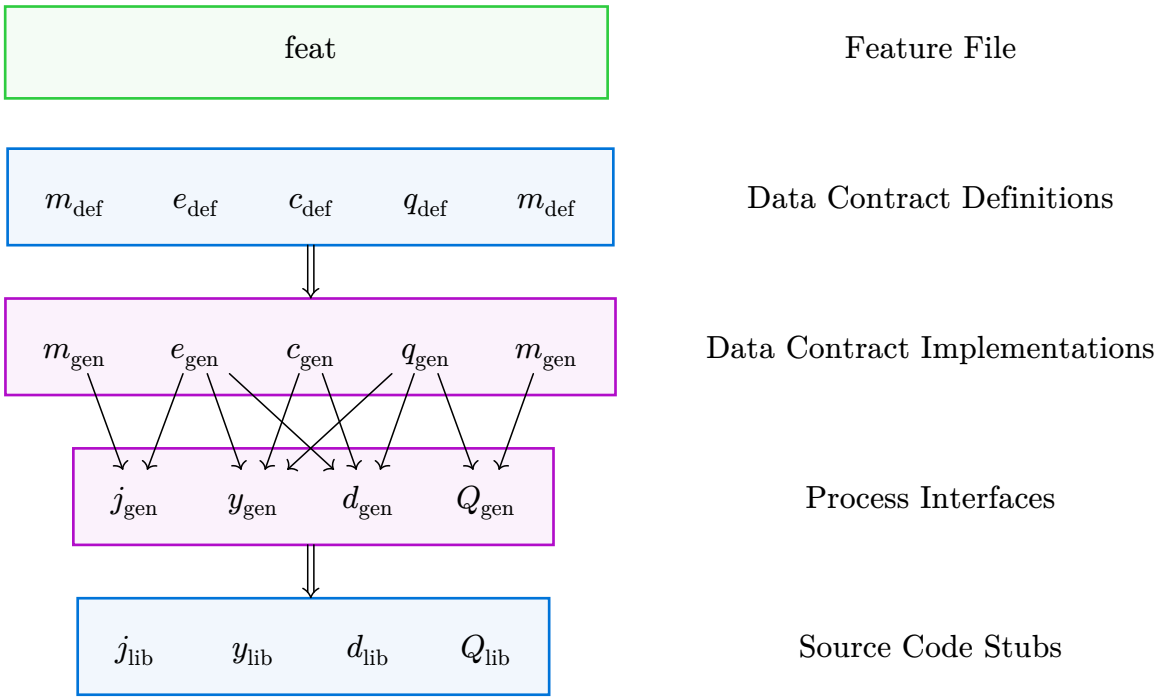


Figure 13: The DIZZY generation pipeline. A Feature File (green) is authored during domain discovery. Data Contract Definitions (blue) are then populated with field-level detail. The generator compiles these into typed Data Contract Implementations and Process Interfaces (purple), which are packaged as runtime-specific library artifacts (blue) that engineers implement against.

The outcome of workshopping is a *Feature File*. The *Feature File* is a structured declaration of every component the system requires: which Commands exist, which Procedures handle them, which Events are then produced, which Policies react, which Projections maintain which Models, and which Queries serve those Procedures and Policies.

The *Feature File* is a map. It documents the high level structures which are then resolved into LinkML schemas and language-specific interfaces.

A developer, a technical lead, or the DIZZY automation tool can look at a Feature File and list exactly what needs to be built. Each Procedure and Policy is a bounded unit of work with explicitly declared inputs, outputs, and data dependencies.

The generation pipeline then takes this map and produces the scaffolding that engineers implement against.

4.3.1. Building Data Definition Schemas

The Feature File names components but does not describe the shape of the data they carry. Building data definitions is the step that gives those names their structure. What fields does a "receipt ingested" event carry? What is the schema of the "receipts" model? These questions are answered by authoring definitions in LinkML - a language-agnostic schema language that represents domain objects in YAML, independent of any particular programming language, framework, or runtime. The Feature File is used to generate LinkML stubs for every Data Definition that is required, details can be filed in after.

4.3.2. Building Program Processes

After Data Definitions are built, and the component wiring declared in the Feature File, typed interfaces are generated for every process. The interface for a Procedure declares exactly which Command it receives, which Queries it is permitted to call, and which Events it may emit. The interface for a Policy declares the Event that triggers it and the Commands it is allowed to dispatch.

At this stage, the The Feature File becomes a compile-time constraints for the architecture. An implementation that exceeds its declared scope is a type error, and can be caught before deployment rather than after.

This generation step makes an explicit claim: the hard architectural decisions have already been made. By the time an engineer receives a generated interface, the question of what each component does - and what it is forbidden from doing - has been resolved at the domain level. What remains is writing the Interstitial Infrastructure to tie the component flows together. The interface enforces the boundary between domain thinking and implementation.

4.3.3. Packaging and Using Libraries

The final step packages the generated interfaces and their compiled data types into importable library artifacts. A Procedure in Python receives a Python library. A Policy in Rust receives a Rust library. A Querier in TypeScript receives a TypeScript library. Each library contains the same domain contracts, as generated by LinkML.

This is the handoff between the Domain and the Deployment.

Engineers implement against the library. The library does not change unless the domain definitions change. Infrastructure decisions, such as which database backs a Model, which message queue carries a Command - are made after the library is built, and they are made at a different layer of the architecture. Domain logic never knows where its inputs came from or where its outputs go.

5. Conclusion

For the formal specification of DIZZY components, schemas, and code generation pipeline, see the companion *DIZZY Specification*.

6. Appendix

6.1. Common Patterns

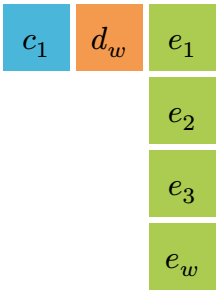
6.1.1. Durable Execution

A procedure can *preform work* if and only if the work has never been started, or in the cases where a previous fault or outage interrupted the runtime.

An explicit failure-as-ended state helps to disambiguate between runtime failures of the procedure and external failures like network interruption, power loss, etc.

For some procedure d_w a durable version d_d is a procedure that wraps d_w such that

1. **Durable Execution Procedure** queries for **Activity Started**
    1. If no **Activity Started**, then emit **Activity Started**
    2. If any **Activity Started**, then query for **Activity Ended** on ID
        1. If no **Activity Ended** - do d_w.
        2. If any **Activity Ended** - return



Symbol	Component	Role
c_1	Command	Start Activity with ID
d_d	Procedure	Durable Execution Procedure
e_1	Event	Activity Started
e_2	Event	Activity Ended (Succeeded)
e_3	Event	Activity Ended (Failed)
e_w	Event	Existing d_w events

6.1.2. Provenance

- Activities
- Entities
- Agents (Human and LLM)

Bibliography

[1] Standish Group, "CHAOS Report 2015," technical report, 2015. [Online]. Available: https://www.standishgroup.com/sample_research_files/CHAOSReport2015-Final.pdf

[2] M. E. Conway, "How Do Committees Invent?," *Datamation*, Apr. 1968, [Online]. Available: http://www.melconway.com/Home/Committees_Paper.html

[3] M. Skelton and M. Pais, *Team Topologies*. IT Revolution Press, 2019.

[4] A. Brandolini, *Introducing EventStorming*. Leanpub, 2021. [Online]. Available: https://leanpub.com/introducing_eventstorming